

School of Informatics



Computer Vision Mini-Project Image segmentation - The Oxford-IIIT Pet Dataset

September 2025

Abstract

This paper presents a systematic comparison of image segmentation techniques applied to the Oxford-IIIT Pet Dataset. We evaluate three distinct architectures: U-Net, pre-training with an Autoencoder, and CLIP feature-based. Finally, modifying the latter, we performed prompt-based interactive segmentation. Through several experiments, we demonstrate that CLIP outperforms the rest of the models, achieving IoU 0.85 and Dice 0.92. Furthermore, we conducted extensive robustness analysis by applying eight perturbation types of increasing intensity to our best-performing model. The results prove that the model maintains consistent performance under most perturbations, with moderate decrease under certain conditions. The proposed prompt-based framework enabled object segmentation with minimal user interaction, showing its potential for practical applications through an implemented UI. This study provides valuable insights into the relative merits of different strategies and their robustness characteristics for pet image segmentation tasks.

Teaching team: Laura Sevilla, Eleonora D'Arnese, Changjian Li

1 Introduction

Image segmentation is a core task in computer vision that involves dividing an image into regions by labelling each pixel. In this report, we investigate image segmentation using the *Oxford-IIIT Pet Dataset*, a widely recognized dataset featuring images of pets of various breeds, each accompanied by pixel-wise segmentation masks. Our study explores various deep learning methods for the task of pixel-wise classification.

We begin by preprocessing the dataset, resizing images to a uniform 256x256x3 resolution and normalizing pixel values, while using data augmentation techniques such as geometric transformations, colour adjustments, and noise perturbations to increase model robustness and generalization (Alomar et al. 2023) [1]. Next, we design and train four distinct segmentation models: a U-Net architecture as a reliable baseline, an Autoencoder-based model that uses unsupervised pre-training for feature extraction; a CLIP-based model that utilizes the CLIP pre-trained encoder from a large-scale vision-language model (Radford et al. 2021) [2]; and a prompt-based segmentation model that adapts the top-performing architecture to incorporate user-provided prompts for interactive segmentation. The first three were trained to predict three distinct classes per pixel (dog, cat and background), while the last model was intended for binary predictions (region of interest, none). We then evaluate these models using metrics like Mean Intersection over Union (IoU), Dice coefficient, and accuracy, analyzing the contributions of pre-training and augmentation in various combinations. Finally, we test the robustness of the best-performing model by applying various levels of perturbations to test set images and studying their impact on segmentation.

This report is organized as follows: Section 2 covers the dataset content, preprocessing, and augmentation methods; Section 3 details the design and implementation of the segmentation models; Section 4 describes the experimental setup for training and evaluation; Section 5 presents the performance evaluations of different augmentation techniques across all models, along with a discussion of such results; Section 6 investigates robustness under perturbations for the best performing model; finally, the Conclusions 7 recaps key findings and proposes future research directions.

2 Dataset and Augmentations

2.1 Dataset

The dataset used in this study is the *Oxford-IIIT Pet Dataset*, a well-known benchmark for image segmentation and classification tasks. This dataset consists of 7,349 RGB images of 37 different pet breeds, including their corresponding segmentation masks. Each image is accompanied by two types of annotations: a classification label indicating the breed and a pixel-wise segmentation mask outlining the pet’s silhouette. The dataset was loaded using TensorFlow Datasets, which eased the process of obtaining the data in a structured form. The dataset is split by default into training and test sets in a 90%-10% split. The test set was left untouched, while 10% of the default training set was used to validation.

2.2 Preprocessing

To prepare the dataset for training, several preprocessing steps are outlined, as shown in C.1.

1. **Resizing and Normalization:** All images were resized to a fixed shape of (256, 256) using bilinear interpolation, and pixel values were normalized to the range [0, 1]. These changes were made to ensure consistency throughout the different models input and to improve numerical stability during training.

2. **Segmentation Mask Adjustments:** Since the dataset includes segmentation masks with different labels for background, animal, and borders, the following modifications were applied:

- The border class was merged into the background class.
- Target masks were adjusted based on the species category, splitting the classes of cats and dogs, leaving three classes in total, 0 for background, 1 for dogs and 2 for cats.
- For the prompt-based segmentation task, Gaussian heatmaps were generated for random points to provide guidance to the model. Additionally, separate masks were created specifically for this task by reducing the original three-class segmentation to a two-class format (region of interest, none) based on the selected pixel. These binary masks are used to enforce the model to only predict the user selected region (more details are provided in 3.4).
- Perturbations options were applied to the test set only, to investigate the robustness of the best model (more details in 6).

2.3 Data Augmentation

To enhance the model’s generalization and robustness, data augmentation techniques were applied. These methods introduced diverse modifications to the dataset, helping to reduce overfitting, as discussed in the study by Alomar et al. 2023 [1]. The augmentations were implemented using TensorFlow and Keras [3, 4] and grouped into three main categories: geometric transformations, colour-based adjustments, and noise-based perturbations. Additionally, various combinations of these techniques were tested to further increase dataset variability and improve model performance.

2.3.1 Geometric Transformations

Geometric augmentations modified image orientation and positioning to increase spatial diversity. Random flipping flips images horizontally with a 50% probability, while random rotation introduces small angular shifts of $\pm 2^\circ$. Random translation shifts images by up to 10% in both directions, filling missing pixels with a constant value. For segmentation tasks, transformations were also applied to the corresponding masks, while ensuring label integrity (pixels are still assigned to the right class) (see Appendix 1 for details).

2.3.2 Colour-Based Transformations

Colour augmentations help simulate different lighting conditions and environmental variations by adjusting RGB pixel values. Brightness adjustments modify brightness by up to 20%, while contrast adjustments vary contrast within a 90% to 110% range. Saturation is adjusted within a 0.5 to 1.5 scaling range, and hue variations shift values up to $\pm 18^\circ$. These transformations were applied only to the image, ensuring that segmentation masks remained unchanged (see Appendix 2 for implementation).

2.3.3 Noise and Filtering

To improve robustness, noise-based perturbations and compression artifacts were introduced. Gaussian noise was added with a standard deviation of 0.1, while custom filtering kernels applied

random Gaussian or motion blur. Additionally, JPEG compression was simulated by encoding images at varying quality levels (30-70), introducing typical artifacts such as blocking and blurring (see Appendix 3 for details).

2.3.4 Implementation Details

Augmentations were applied dynamically during training, ensuring diverse inputs without altering the dataset. Each augmentation was tested separately by applying it to every batch in an experiment (see Appendix 5). The *augment_image_mask* function kept transformations consistent between images and masks, while the *batch* function handled augmentation selection and applying dynamic batching to the training set. It works by caching the raw data, loading and augmenting it for each batch (see Appendix 6).

To maintain label consistency, geometric transformations were applied to both images and masks, while Colour and Noise augmentations were applied only to images. After transforming the image and mask, the mask values were filtered to keep only the original class labels. Each mask contains two distinct values: (0,1) for background and dog or (0,2) for background and cat, ensuring the labels remain unchanged.

Figure 1 shows the effects of different augmentation techniques on an image and its corresponding segmentation mask. Full implementation details can be found in Appendix 4.



Figure 1: Example of data augmentations applied to an image and its segmentation mask.

3 Segmentation networks design and implementation

3.1 U-Net end-to-end segmentation

In 2015, Ronneberger et al. [5] proposed the U-Net architecture, which has become one of the most influential models for image segmentation tasks.

As shown in Fig. 9 in Appendix B.2, the U-Net architecture is characterized by its distinctive U-shaped design, consisting of a contracting path (encoder) that captures contextual information and a symmetric expanding path (decoder) that recovers fine-grained spatial features and defines object boundaries. This structure allows the network to propagate context information to higher resolution layers through skip connections that concatenate features from the contracting path with those in the expanding path. The model demonstrated a remarkable increase in performance by incorporating data augmentation techniques. Indeed, for our task, the model achieved optimal performance when implementing two specific data augmentation techniques (Colour + Geometric) from the original U-Net methodology, as shown in Table 1.

The U-Net architecture consists of four downsampling blocks (each consisting of two 3×3 con-

volutional layers followed by a 2×2 max pooling operation) and four upsampling blocks (each using 2×2 transposed convolutions followed by two 3×3 convolutional layers). The network preserves the original paper channel counts of [64, 128, 256, 512, 1024] across the different levels and uses ReLU activation functions throughout. Following established practices in modern U-Net variants [6, 7], we used 'same' padding which preserves spatial dimensions throughout the network, eliminating the need to crop feature maps before concatenation and thereby simplifying the implementation of skip connections. For upsampling, we implemented transposed convolutions, following the approach of Long et al. [8] and further refined in encoder-decoder architectures like SegNet (Badriana et al. 2017 [9]). Finally, the output layer is configured with 3 output classes using a softmax activation function to produce probability maps for each class.

3.2 Autoencoder pre-training for segmentation

Autoencoders are neural networks that learn to compress and reconstruct data without needing labeled examples. In this project, we used an autoencoder to pre-train an encoder on the raw images, helping it to identify key patterns before applying it to image segmentation. This method allows the network to learn useful features from unlabeled images first, making the segmentation task easier and more effective.

3.2.1 Autoencoder pre-training for segmentation

The Autoencoder, built using TensorFlow and Keras [3, 4], consists of two main parts: an encoder and a decoder. The encoder includes five convolutional layers with increasing sizes: 64, 128, 256, 512, and 1024. Each convolutional layer applies a set of learnable filters to the input image, extracting spatial patterns such as edges, textures, and more complex structures as depth increases. Following each convolutional layer, max-pooling operations reduce the image's spatial dimensions by half, using 'same' padding to maintain dimensions before pooling. This hierarchical configuration compresses the input into a compact latent representation with 1024 feature maps, preserving key structural details. The decoder then mirrors the encoder's design, reconstructing the original image through transposed convolutional layers paired with upsampling. A final sigmoid activation layer ensures that the output matches the original dimensions (see Appendix 7). This architecture aligns with the proposed encoder design and the multiscale techniques of Yusiong and Naval 2022 [10], which enable effective reconstruction.

The training performance of the Autoencoder, including training and validation loss curves over 100 epochs, is presented in Figure 7 Appendix B.1.1. A qualitative example illustrating an input image alongside its reconstructed output is shown in Figure 8 in Appendix B.1.2. Finally, the Autoencoder's test performance evaluation is reported in Table 2 in Appendix B.1.1.

3.2.2 Transfer to Segmentation Model

Once the Autoencoder was trained, we reused its encoder's weights in a segmentation model. The segmentation decoder applies four transposed convolutional layers (512, 256, 128, and 64 filters) with upsampling to restore the original image size, concluding with a softmax layer to classify the image into three pixel categories (see Appendix 8). The pre-trained weights of the Autoencoder are integrated and used by the segmentation model for this task. This model benefits from the pre-trained weights, accelerating the learning process and enhancing generalization, as analyzed by Yusiong and Naval (2022) [10].

3.3 CLIP features for segmentation

The CLIP model, introduced by Radford et al. 2021 [2], learns a shared embedding space for images and text, enabling remarkable performance in tasks such as classification and zero-shot learning (classification of images by using text descriptions, e.g., *"a photo of a dog"*). Recent studies, like Zhou et al. 2022 [11], have further adapted CLIP’s pre-trained Vision Transformer (ViT) encoder specifically for image segmentation tasks. By utilizing the semantic-rich features learned by the encoder and freezing its pre-trained weights, they’ve significantly reduced the amount of labeled data required. This method accelerates the training process and improves performance on diverse and unseen images, making it effective for segmenting standard-sized images.

We use the pre-trained *"openai/clip-vit-base-patch32"* encoder [12], implemented with TensorFlow and Keras [3, 4], to segment the images. Initially, the input images are resized to $224 \times 224 \times 3$ using a Resizing layer, followed by normalization with CLIP’s mean values ([0.48145466, 0.4578275, 0.40821073]) and standard deviation values ([0.26862954, 0.26130258, 0.27577711]) through a custom Lambda layer, ensuring compatibility with the encoder’s pre-trained weights (see Appendix 9). The `CLIPEncoderLayer`, with a frozen (non-trainable) encoder, changes the data format from NHWC (batch, height, width, channels) to NCHW (batch, channels, height, width) using `tf.transpose`, producing a feature output of shape (batch_size, 50, 768). This consists of 49 spatial patches arranged in a 7×7 grid and one class token, each represented by a 768-dimensional vector.

The class token is removed using a Lambda layer, and the remaining data is reshaped to (batch_size, 7, 7, 768) with a Reshape layer. The decoder subsequently upsamples these features using a sequence of Conv2D layers with decreasing filter sizes (256, 128, 64, 32) and 3×3 kernels, combined with UpSampling2D layers that double the spatial dimensions at each step. Segmentation is finalized with a 1×1 Conv2D layer containing 3 filters and Softmax activation, resulting in the desired $256 \times 256 \times 3$ segmentation mask (see Appendix 10 for more details).

3.4 Prompt-based segmentation

Prompt-based segmentation enables users to provide guidance to a model by specifying a particular region of interest within an image. The current implementation was designed to deliver the following functionality via a UI: a user views an image displayed on the screen and selects a specific pixel location within it. Based on this input, the system predicts a binary mask—classifying each pixel as either part of the "region of interest" or "none"—and subsequently displays the resulting mask to the user.

To achieve this functionality, the program internally generates a heatmap corresponding to the selected pixel location. Both the original image and the generated heatmap are then passed as inputs to the model, which outputs the predicted segmentation mask.

Several steps were necessary to adapt the data for this task, especially to include the augmentation techniques consistently.

3.4.1 Data processing

For training, we randomly sampled pixel coordinates within the image (the user will later be able to select them manually on the UI). Then, they were used to generate a Gaussian heatmap, matching the image dimensions. Following the data preprocessing steps of Section C.1, the target masks were binarized, assigning a value of 1 to pixels within the region of interest and 0 to all others (see Appendix H.1 for the functions used). For the validation and test sets, these

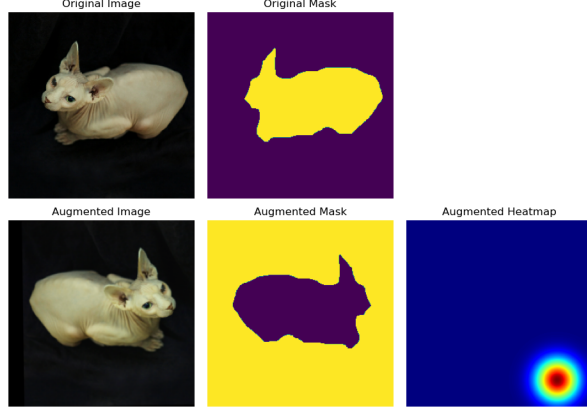


Figure 2: Example of heatmap generation on the training set. After data augmentation, the heatmap is generated and the new mask reflects to the desired values (1 for the region of interest and 0 otherwise).

steps were applied during preprocessing (see C.1). However, the training set required a different approach. Applying data augmentation after heatmap generation represented a significant challenge. Specifically, some of the Geometric transformations (rotations and translations) introduce additional margin regions. Filling the additional frames with constant values (as done in the previous tasks) was insufficient because the Gaussian heatmap’s values vary largely per pixel, based on their distance from the central coordinates. Consequently, the processes of random coordinate generation, heatmap creation, and target mask adjustment were integrated into the dynamic augmentation pipeline (see H.2 and 6). An example of an augmented result is shown in Fig. 2.

3.4.2 Prompt-CLIP Model

To accommodate the extra input of the heatmap, the model was modified (H.3) to accept a four-channel input instead of the original three-channel RGB image, updating the input shape from (256, 256, 3) to (256, 256, 4), where the additional channel represents the single-channel heatmap. The first step of the model is separating the image from the heatmap. This is necessary to ensure the RGB image follows the previous preprocessing pipeline—resizing to 224x224, normalizing, and passing through the CLIP encoder to extract visual features. Meanwhile, the heatmap is processed separately through a small encoder network: it is resized to 28x28, fed through two convolutional layers with max pooling in between to capture spatial features, and then processed by a 1x1 convolutional layer to project it to 768 channels, followed by resizing to 7x7. These heatmap features are concatenated with the CLIP features (reshaped to 7x7x768) along the channel axis to re-concatenate both visual and spacial information. The combined feature map is then passed through the decoder, consisting of Conv2D and UpSampling2D layers, to produce a single-channel segmentation mask with sigmoid activation.

3.4.3 UI

Finally a simple user interface (UI) was implemented to demonstrate a practical application of the point-based segmentation model. The interface allows users to load an image and select an object of interest with a click. Each click triggers the generation of a Gaussian heatmap centered at the chosen point, which acts as a prompt for the model to predict the segmentation mask of the corresponding object. The resulting mask is then overlaid onto the original image.

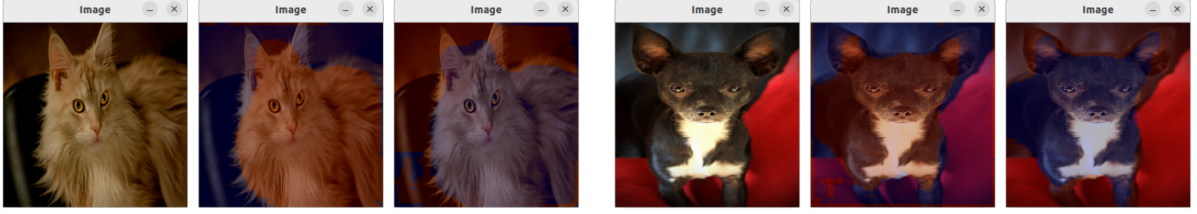


Figure 3: UI screenshots for two sample images, showing three interaction steps: (1) initial view with no user input, (2) segmentation result after selecting an animal pixel (animal highlighted in red, background in blue), and (3) updated segmentation after selecting a background pixel, isolating the background region.

The region of interest is highlighted in red and the rest in blue. Fig. 3 illustrates the UI with a series of screenshots, while the code can be found in H.4

4 Experimental setup

Four segmentation models were evaluated: U-Net, an Autoencoder-based model, a CLIP-based feature extractor, and a prompt-based CLIP segmentation model. Training images were batch-augmented (batch size 32), while validation images remained unchanged (see Appendix 5) .

All models were trained using the Adam optimizer (learning rate $1e-4$) for up to 100 epochs with early stopping (patience 20 epochs). The best model checkpoint was selected based on validation loss. U-Net, the Autoencoder-based model, and the CLIP-based segmentation model used sparse categorical cross-entropy loss, while the prompt-based model used binary cross-entropy (given that the three classes are reduced to two). The Autoencoder was first pre-trained with mean squared error (MSE) before segmentation fine-tuning (see Appendix 11, 12).

Performance was evaluated using Mean IoU, Dice coefficient, and accuracy. Mean IoU measures the average intersection over the union of predicted and ground-truth segments across all classes, providing an overall segmentation quality score. The Dice coefficient quantifies the overlap between predicted and actual segmentation masks, emphasizing precision and recall balance (see Appendix 13). Accuracy was used for the Autoencoder during pre-training to assess reconstruction consistency . Detailed training metrics and performance plots are provided in Appendix B, depicted in Figure 6.

5 Performance evaluation

This section evaluates and discusses the performance of the four segmentation models across varying augmentation strategies.

5.1 Overall Model Comparison

The CLIP-based model significantly outperformed other architectures, achieving a Dice score of 0.92 and IoU of 0.85 under combined Colour and Geometric augmentations (C+G), surpassing U-Net (Dice: 0.83, IoU: 0.53) and the Autoencoder (Dice: 0.78, IoU: 0.66) (see Table 1). This difference is due to CLIP’s pre-trained Vision Transformer (ViT) encoder, which uses large-scale training to extract semantically rich features. Unlike U-Net and Autoencoder, which were trained exclusively on our specific task, CLIP was trained on a dataset with 400 million image-text pairs. This large and diverse training set allows CLIP to extract good high-level features, even in our limited context. Another advantage is ViT’s global self-attention mechanism, which is particularly useful for segmenting uncommon shapes and complex backgrounds (Radford et

al., 2021 [2]).

The U-Net lower performance can be justified by its high sensitivity to class imbalances. As discussed by Isensee et al. 2021 [13], its encoder-decoder structure have difficulties with fine-grained boundary identification when background pixels dominate ($\sim 80\%$ of masks), leading to under-segmentation in cases of occluded or small pets. The Autoencoder’s pre-training phase, though reaching comparable results to U-Net, lacked semantic understanding provided by CLIP’s pre-training, as its reconstructive objective emphasizes low-level features (edges, textures) over class-specific patterns. We note though that integrating the pre-train encoder from the Autoencoder into the segmentation model, the training process was more effective, minimizing the validation loss with less epochs (~ 20).

The Prompt-CLIP model, adapted for interactive segmentation, achieved a lower IoU (0.66) than its non-prompt counterpart. This was expected, because the binary segmentation task was inherently more ambiguous. In this case the model must interpret user’s intent from sparse Gaussian signals.

5.2 Impact of Data Augmentation

Augmentation techniques affected model performance in multiple ways:

- Geometric augmentations (G) most benefited U-Net (Dice: +21% vs. base), aligning with its spatial design (mostly using convolutions and MaxPooling).
- Colour augmentations (C) slightly reduced CLIP performance (Dice: -0.07% vs. base), likely because its pre-training already provides robustness to variations in illumination, while U-Net showed a modest improvement (Dice: +2.8%).
- Combined C+G maximized performance for all models, with CLIP achieving maximum IoU (0.85). This showed that combining both spatial and visual diversity helps improve generalization.
- Adding noise (N) to the C+G setup reduced performance compared to the best (C+G) augmentation strategy. The CLIP Dice drop by -0.62%, may be due to noise that disrupts local spatial features crucial for segmentation. The Dice of U-Net dropped more sharply, approximately -4.7%, indicating a weaker robustness to random noise.

5.3 Challenges Affecting Segmentation Mask Quality

Segmentation models perform significantly poorly when objects (such as cats or dogs) occupy a pixel area that is substantially smaller than the background, leading to frequent misclassifications where the object pixels are incorrectly identified as background (Figure 4b). On the other hand, when the object occupies a larger or comparable pixel area to the background, segmentation accuracy improves significantly, with masks aligning closely to the ground truth (Figure 4a). However, even successful segmentations can miss fine details, such as ears or object boundaries, due to model limitations in spatial resolution. Potential difficulties include variable lighting conditions, occlusions, and similar textures between objects and their backgrounds.

6 Robustness exploration

Our best-performing CLIP-based segmentation model—trained using a combination of color and geometric augmentations—was evaluated for robustness against eight incremental test-time perturbations (see Appendix C.1 for details). These included Gaussian noise ($\sigma \in [0, 18]$), blurring (iterative 3×3 averaging kernels), brightness and contrast shifts (via multiplicative or additive scaling), occlusion (random black squares), and salt-and-pepper noise (random pixel

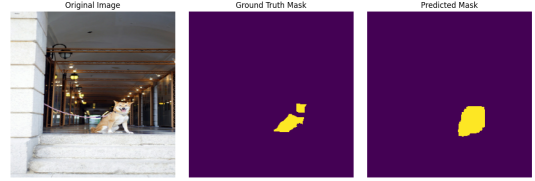
Aug Type	UNet				Autoencoder			
	Acc	Dice	Loss	IoU	Acc	Dice	Loss	IoU
Base	0.8404	0.6775	0.4060	0.5561	0.8568	0.7587	0.3963	0.6343
C	0.8352	0.6965	0.4142	0.5676	0.8557	0.7434	0.3817	0.6178
G	0.8965	0.8226	0.3136	0.7131	0.8621	0.7540	0.3526	0.6275
C + G	0.9003	0.8321	0.2852	0.5254	0.8714	0.7768	0.3469	0.6557
C + G + N	0.8813	0.7931	0.3314	0.6748	0.8604	0.7555	0.3917	0.6297

Aug Type	CLIP				Prompt-CLIP			
	Acc	Dice	Loss	IoU	Acc	Dice	Loss	IoU
Base	0.9317	0.9081	0.1725	0.8350	—	—	—	—
C	0.9312	0.9075	0.1652	0.8342	—	—	—	—
G	0.9387	0.9179	0.1530	0.8506	—	—	—	—
C + G	0.9398	0.9188	0.1500	0.8520	0.8115	0.8530	0.3914	0.6623
C + G + N	0.9353	0.9131	0.1577	0.8425	—	—	—	—

Table 1: Comparison of Test Performance Across Different Models with Augmentations. Each model’s Accuracy (Acc), Dice Score (Dice), Loss, and Intersection over Union (IoU) are reported. ‘Base’ refers to No Augmentation, ‘Single Augmentations’ include Colour Augmentations (C) and Geometric Augmentations (G), while we also have combinations of augmentations: Colour + Geometric (C + G) and Colour + Geometric + Noise (C + G + N).



(a) Successful prediction: the model accurately segments the majority of the pet, but fails for complex edges (ears).



(b) Failed prediction: the model misses or incorrectly segments parts of the pet.

Figure 4: Comparison of model predictions for segmentation. Left: a correctly predicted mask. Right: an example of failure due to class imbalance (the majority of pixels assigned to the background compared to the dog class).

replacement). As shown in Fig. 5, the model demonstrated strong resilience, with performance degrading by less than 3% for most distortions even at maximum intensity.

Under extreme contrast reduction (e.g., scaling factor of 0.1), the model showed significant drops in accuracy (-10%), Dice (-25%), and IoU (-28%), due to the diminished visual distinction between objects and their surrounding regions. Salt-and-pepper noise caused a gradual performance decline (up to -9% accuracy, -8% Dice), likely due to the absence of such noise patterns in the training data. In contrast, the model remained robust to moderate variations in brightness, contrast, and occlusion, supported by training augmentations such as affine transformations and color jitter. Sensitivity to severe or structured distortions reflects limited exposure to these conditions during training. Overall, the model’s robustness is attributed to the CLIP encoder’s extensive pretraining on diverse visual data and the decoder’s adaptation to augmented inputs, highlighting the importance of training diversity in achieving reliable performance under real-world perturbations.

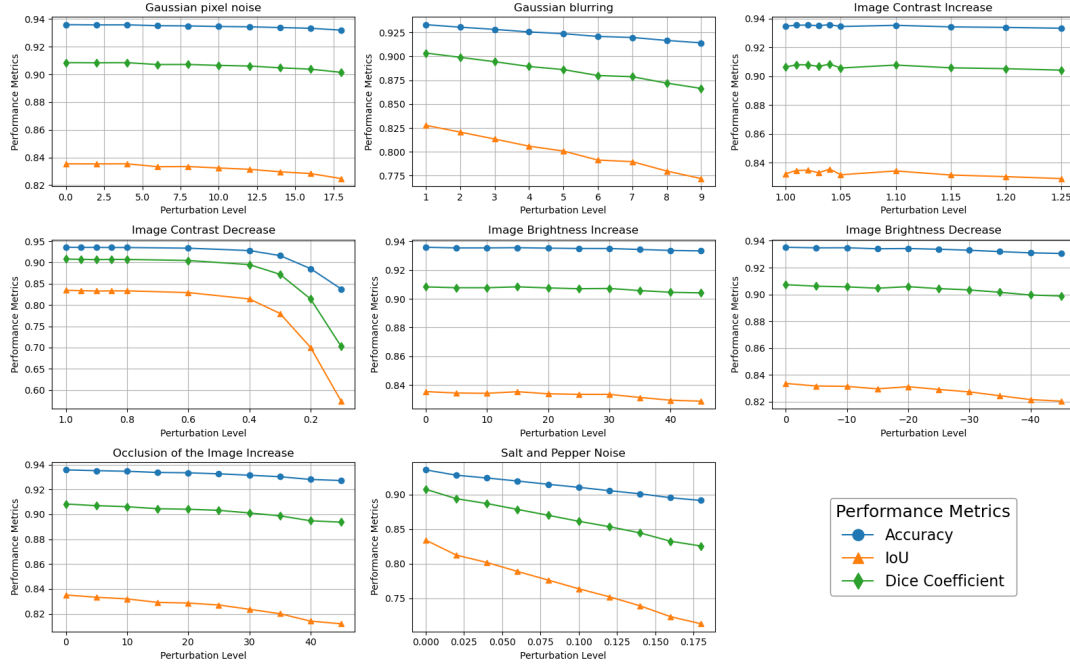


Figure 5: Perturbation plots of robustness exploration for the best performing model (CLIP Model with Colour + Geometric augmentation). Perturbation levels on the x-axis, performance metrics on the y-axis.

7 Conclusions

This study systematically evaluated four deep learning approaches for pet image segmentation on the Oxford-IIIT Pet Dataset: U-Net, autoencoder pre-training, CLIP feature-based segmentation, and prompt-based CLIP segmentation. A functional user interface was also developed for the latter, demonstrating its potential in practical, interactive scenarios.

Among the evaluated models, the CLIP-based approach achieved the highest scores, with an IoU of 0.85 and a Dice score of 0.92 when trained using both Colour and Geometric augmentations. These results reflect the benefits of CLIP’s large-scale pretraining and the Vision Transformer’s ability to extract meaningful semantic features.

Data augmentation generally improved segmentation performance. Geometric transformations were particularly effective for U-Net (+18% Dice), while CLIP did not benefit from Colour augmentations, suggesting that its pretraining already provided robustness to such variations. The combination of Colour and Geometric augmentations proved to be the most effective strategy across models.

In robustness analysis, the CLIP-based model maintained consistent performance across most perturbations. However, it showed sensitivity to extreme contrast reductions and structured noise, likely due to limited exposure to such variations during training.

Two key limitations emerged: class imbalance (background pixels dominated $\sim 80\%$ of masks) and dataset specificity. Future work should explore focal loss [14] or reweighting strategies to address bias toward dominant classes and validate findings on more diverse datasets like *COCO* [15]. Additionally, hybrid architectures—such as ViT-U-Net fusion—could marry CLIP’s semantic richness with U-Net’s localization precision. For interactive segmentation, multi-click prompts, textual guidance, or attention mechanisms may reduce ambiguity in user inputs.

Bibliography

- [1] Khaled Alomar, Halil Ibrahim Aysel, and Xiaohai Cai. Data augmentation in classification and segmentation: A survey and new strategies. *Journal of Imaging*, 2023.
- [2] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. *arXiv preprint arXiv:2103.00020*, 2021.
- [3] Google Brain Team. Tensorflow: Machine learning framework. <https://www.tensorflow.org/>, 2015.
- [4] F. Chollet and Others. Keras: Deep learning for python. <https://keras.io/>, 2015.
- [5] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation, 2015.
- [6] Özgün Çiçek, Ahmed Abdulkadir, Soeren S Lienkamp, Thomas Brox, and Olaf Ronneberger. 3d u-net: Learning dense volumetric segmentation from sparse annotation. In *Medical Image Computing and Computer-Assisted Intervention–MICCAI 2016*, pages 424–432. Springer, 2016.
- [7] Fabian Isensee, Paul F Jaeger, Simon AA Kohl, Jens Petersen, and Klaus H Maier-Hein. nnu-net: a self-configuring method for deep learning-based biomedical image segmentation. *Nature methods*, 18(2):203–211, 2021.
- [8] Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3431–3440, 2015.
- [9] Vijay Badrinarayanan, Alex Kendall, and Roberto Cipolla. Segnet: A deep convolutional encoder-decoder architecture for image segmentation. *IEEE transactions on pattern analysis and machine intelligence*, 39(12):2481–2495, 2017.
- [10] J. P. T. Yusiong and P. C. Naval. Multi-scale autoencoders in autoencoder pre-training for segmentation. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.
- [11] Ziqin Zhou, Bowen Zhang, Yinjie Lei, Lingqiao Liu, and Yifan Liu. Zegclip: Towards adapting clip for zero-shot semantic segmentation. *arXiv preprint arXiv:2212.03588*, 2022.
- [12] OpenAI and Hugging Face. openai/clip-vit-base-patch32. <https://huggingface.co/openai/clip-vit-base-patch32>, 2021. Accessed: 2025-03-18.
- [13] Fabian Isensee, Paul F. Jaeger, Simon A. A. Kohl, Jens Petersen, and Klaus H. Maier-Hein. nnu-net: a self-configuring method for deep learning-based biomedical image segmentation. *Nature Methods*, 18(2):203–211, 2021.
- [14] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE international conference on computer vision*, pages 2980–2988, 2017.

- [15] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. Microsoft coco: Common objects in context. In *Computer vision–ECCV 2014: 13th European conference, zurich, Switzerland, September 6–12, 2014, proceedings, part v 13*, pages 740–755. Springer, 2014.

A Appendix: Workload Management

Description of each member’s responsibilities.

Ioannis Vdias (s2713719):

- Autoencoder pre-training model.
- CLIP-based model (segmentation).
- Training and Testing models (GPU usage).
- Robustness evaluations under perturbations (CLIP best performing model).

Alessandro Costanzo Ciano (s2659893):

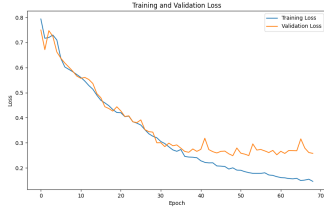
- Data loading and preprocessing.
- U-Net model.
- Prompt (user guidance) CLIP-based model.
- UI for prompt-segmentation, integrating user’s input.

Common tasks

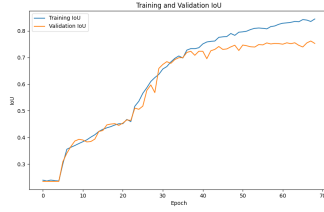
- Data augmentation (functions and implementation).
- Report writing.

B Appendix: Training and Validation Performance of All Segmentation Models

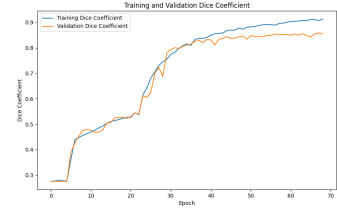
To evaluate model generalization, we present the training and validation performance of all segmentation models under the best-performing augmentation strategy: **Colour+Geometric**. This augmentation consistently improved performance on the test set by increasing robustness to variations in colour and spatial transformations. Each model's performance is visualized below, showing the loss, Mean IoU, and Dice Coefficient metrics over training epochs.



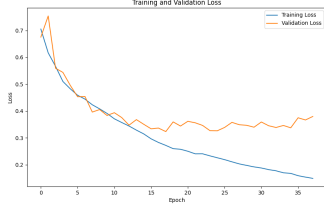
(a) U-Net: Training and Validation Loss



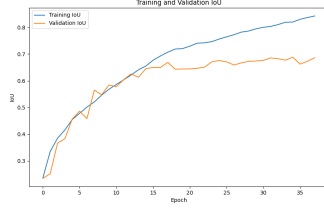
(b) U-Net: Training and Validation IoU



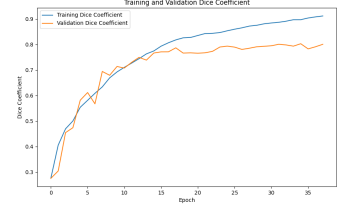
(c) U-Net: Training and Validation Dice Coefficient



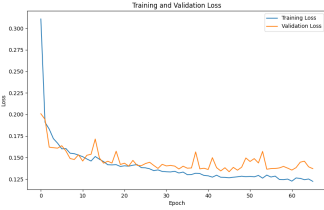
(d) Autoencoder: Training and Validation Loss



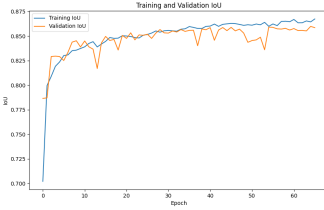
(e) Autoencoder: Training and Validation IoU



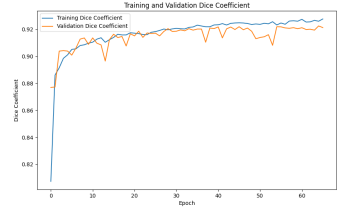
(f) Autoencoder: Training and Validation Dice Coefficient



(g) CLIP: Training and Validation Loss



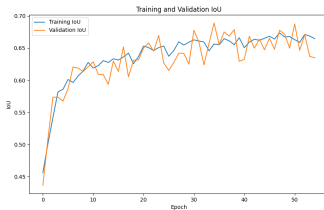
(h) CLIP: Training and Validation IoU



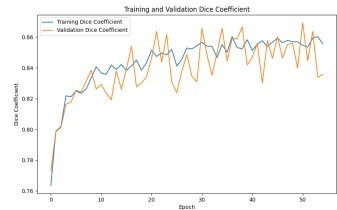
(i) CLIP: Training and Validation Dice Coefficient



(j) Prompt-CLIP: Training and Validation Loss



(k) Prompt-CLIP: Training and Validation IoU



(l) Prompt-CLIP: Training and Validation Dice Coefficient

Figure 6: Training and validation performance metrics for the evaluated segmentation models using the best-performing Colour+Geometric augmentation strategy. Each row corresponds to a model, showing loss, IoU, and Dice coefficient over epochs.

B.1 Autoencoder Architecture

The Autoencoder consists of five convolutional layers in the encoder (with filters: 64, 128, 256, 512, 1024) followed by max-pooling, and a symmetric decoder using transposed convolutions with upsampling. The final layer uses a sigmoid activation to constrain pixel values between 0 and 1.

B.1.1 Autoencoder Training and Validation Loss

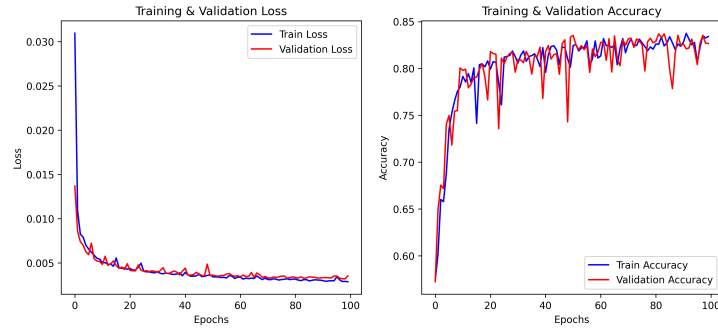


Figure 7: Training and validation loss curves for the Autoencoder over 100 epochs.

Metric	Train	Validation	Test
Loss (MSE)	0.002903	0.003206	0.003529
Accuracy	0.835074	0.835348	0.833299

Table 2: Autoencoder training-test performance metrics (best model selected based on minimum validation loss during training).

B.1.2 Reconstruction Example

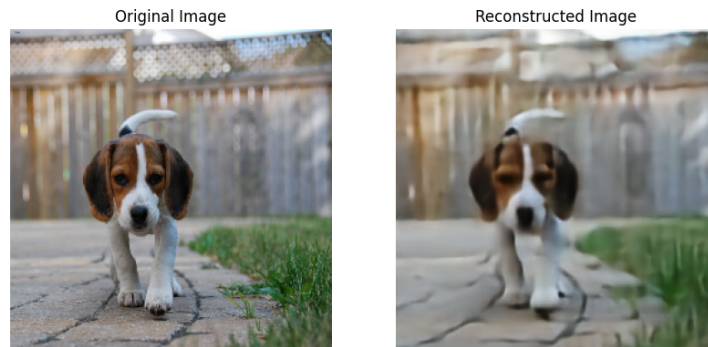


Figure 8: Example comparison between an original image and its reconstructed version by the Autoencoder.

B.2 U-Net architecture

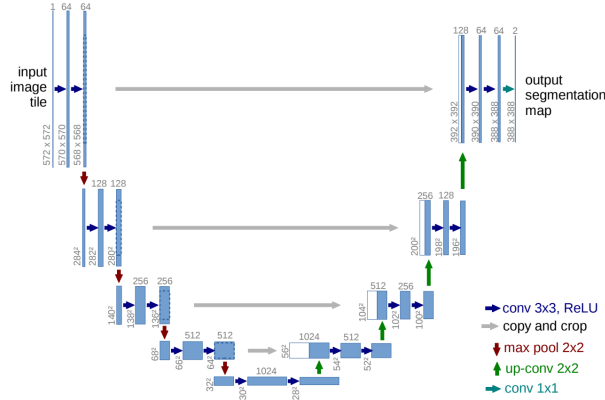


Figure 9: The U-Net architecture (from Ronneberger et al. 2015 [5]).

C Appendix: Code

This appendix contains the source code used in this project.

C.1 Preprocessing Code

Data processing pipeline for the Oxford-IIIT Pet dataset, including loading, preprocessing, mask manipulation, and heatmap generation. It implements six different image perturbation techniques (Gaussian noise, blurring, contrast/brightness adjustments, occlusion, and salt-and-pepper Noise) to test model robustness.

```

1 import tensorflow as tf
2 import tensorflow_datasets as tfds
3 from tensorflow import keras
4 from point_segmentation import random_loc, create_gaussian_heatmap, \
   mask_heat_modification
5
6 class OxfordPetDataset:
7     def __init__(self):
8         self.train_raw = None
9         self.val_raw = None
10        self.test_raw = None
11        self.test_perturbed = None
12        self.ds_info = None
13        self.num_classes = None
14        self.num_train_examples = 0
15        self.num_val_examples = 0
16        self.num_test_examples = 0
17        self.get_label_name = None
18
19    def load_data(self):
20        """Loads the Oxford-IIIT Pet dataset."""
21        (self.train_raw, self.val_raw, self.test_raw), self.ds_info = \
22            tfds.load(
23                name='oxford_iiit_pet',
24                split=['train[:90%]', 'train[90%:]', 'test'],
25                shuffle_files=True,
26                as_supervised=False, # with False each example is a dictionary \
27                                     ['image', 'label', 'segmentation_mask']
28                with_info=True
29            )

```

```

28
29     self.num_train_examples = \
30         tf.data.experimental.cardinality(self.train_raw).numpy()
31     self.num_val_examples = \
32         tf.data.experimental.cardinality(self.val_raw).numpy()
33     self.num_test_examples = \
34         tf.data.experimental.cardinality(self.test_raw).numpy()
35     print(f'Number of training samples: {self.num_train_examples}')
36     print(f'Number of validation samples: {self.num_val_examples}')
37     print(f'Number of test samples: {self.num_test_examples}')
38
39 def res_norm(self, resize_shape=(128, 128)):
40     # Resize and normalize
41     self.train_raw = self.train_raw.map(lambda example: \
42         resize_norm(example, resize_shape))
43     self.val_raw = self.val_raw.map(lambda example: \
44         resize_norm(example, resize_shape))
45     self.test_raw = self.test_raw.map(lambda example: \
46         resize_norm(example, resize_shape))
47
48 def mask_prep(self):
49     # Preprocess mask
50     self.train_raw = self.train_raw.map(lambda example: \
51         mask_preprocessing(example))
52     self.val_raw = self.val_raw.map(lambda example: \
53         mask_preprocessing(example))
54     self.test_raw = self.test_raw.map(lambda example: \
55         mask_preprocessing(example))
56
57 def heatmaps(self):
58     # Generate heatmaps
59     self.val_raw = self.val_raw.map(lambda example: \
60         heatmap_generation(example))
61     self.test_raw = self.test_raw.map(lambda example: \
62         heatmap_generation(example))
63
64 ### Perturbation options ###
65 def first_perturb(self, std=0):
66     self.test_perturbed = self.test_raw.map(lambda example: \
67         first_perturbation(example, std))
68
69 def second_perturb(self, n=0):
70     self.test_perturbed = self.test_raw.map(lambda example: \
71         second_perturbation(example, n))
72
73 def third_perturb(self, a=1.0):
74     self.test_perturbed = self.test_raw.map(lambda example: \
75         third_perturbation(example, a))
76
77 def fourth_perturb(self, b=0.0):
78     self.test_perturbed = self.test_raw.map(lambda example: \
79         fourth_perturbation(example, b))
80
81 def fifth_perturb(self, p=0.0):
82     self.test_perturbed = self.test_raw.map(lambda example: \
83         fifth_perturbation(example, p))
84
85 def sixth_perturb(self, d=0):
86     self.test_perturbed = self.test_raw.map(lambda example: \
87         sixth_perturbation(example, d))

```

```

71
72 #####Resize and normalize#####
73 # single image resize by bilinear interpolation
74 def resize_norm_image(image, resize_shape):
75     image = tf.image.resize(image, resize_shape, \
76                             method=tf.image.ResizeMethod.BILINEAR)
77     image = tf.cast(image, tf.float32) / 255.0
78     return image
79
80 # single mask resize by nearest neighbor
81 def resize_mask(mask, resize_shape):
82     """Resize and normalize a single image."""
83     mask = tf.image.resize(mask, resize_shape, \
84                             method=tf.image.ResizeMethod.NEAREST_NEIGHBOR)
85     return mask
86
87 # resize and normalize a dataset
88 def resize_norm(example, resize_shape):
89     # Resize and normalize
90     seg_mask = example["segmentation_mask"]
91     return {
92         "image": resize_norm_image(example["image"], resize_shape),
93         "label": example['label'],
94         "segmentation_mask": resize_mask(seg_mask, resize_shape),
95         "species" : example["species"]
96     }
97
98 #####Mask preprocessing#####
99 # masks: border -> background, merges border into background
100 def border_to_background(mask):
101     threes_mask = tf.equal(mask, 3)
102     mask = tf.where(threes_mask, tf.cast(2, mask.dtype), mask)
103     return mask
104
105 # masks: cat (1) -> 0
106 def animal_change(mask, species=1):
107     if species == 0:
108         ones_mask = tf.equal(mask, 1)
109         mask = tf.where(ones_mask, tf.cast(0, mask.dtype), mask)
110     return mask
111
112 #background is set to 0 to have consistent filling in geometric \
113 #augmentation with pictures
114 def swap_0_2(mask):
115     twos_mask = tf.equal(mask, 2)
116     zeros_mask = tf.equal(mask, 0)
117     mask = tf.where(twos_mask, tf.cast(0, mask.dtype), mask)
118     mask = tf.where(zeros_mask, tf.cast(2, mask.dtype), mask)
119     return mask
120
121 def mask_preprocessing(example):
122     mask = example["segmentation_mask"]
123     species = example["species"]
124     label = example['label']
125     image = example['image']
126     mask = border_to_background(mask)
127     mask = animal_change(mask, species)
128     mask = swap_0_2(mask)
129     return {

```

```

128         "image": image,
129         "label": label,
130         "segmentation_mask": mask,
131         "species": species
132     }
133
134     #####Heatmap generation ---> Mask modification#####
135     def heatmap_generation(example):
136         image = example["image"]
137         label = example["label"]
138         original_mask = example["segmentation_mask"]
139         species = example["species"]
140         location = random_loc(mask_shape=original_mask.shape)
141         heatmap = create_gaussian_heatmap(mask_shape=original_mask.shape, \
            loc=location, sigma=20)
142         mask = mask_heat_modification(original_mask, location)
143         return {
144             "image": image,
145             "label": label,
146             "segmentation_mask": mask,
147             "original_mask": original_mask,
148             "location": location,
149             "heatmap": heatmap,
150             "species": species,
151             "image+heatmap": tf.concat([image, heatmap], axis=-1)
152         }
153
154     #####Perturbations#####
155     def first_perturbation(example, std=0):
156         '''Gaussian pixel noise'''
157         image = example["image"]
158         shape = tf.shape(image)
159         std = std/255 # normalize to [0, 1]
160         noise = tf.random.normal(shape=shape, mean=0.0, stddev=std)
161         image = tf.clip_by_value(image + noise, 0.0, 1.0)
162         return {
163             "image": image,
164             "segmentation_mask": example["segmentation_mask"]
165         }
166
167     def second_perturbation(example, n=1):
168         '''Gaussian blurring'''
169         kernel = tf.constant([
170             [1., 1., 1.],
171             [1., 1., 1.],
172             [1., 1., 1.]
173         ], dtype=tf.float32) / 9.0
174         # Reshape kernel for 2D convolution
175         kernel = tf.reshape(kernel, [3, 3, 1, 1])
176         image = example["image"]
177         if len(image.shape) == 3:
178             image = tf.expand_dims(image, axis=0) # Add batch dimension
179
180         for _ in range(n):
181             channels = []
182             for i in range(image.shape[-1]):
183                 channel = image[..., i:i+1] # Extract single channel and \
                    keep dimensions
184                 blurred = tf.nn.conv2d(channel, filters=kernel, \
                    strides=[1, 1, 1, 1], padding="SAME")

```

```

185         channels.append(blurred)
186         image = tf.concat(channels, axis=-1)
187
188         image = tf.squeeze(image, axis=0) # Remove batch dimension
189     else:
190         image = tf.expand_dims(tf.expand_dims(image, axis=0), axis=-1)
191
192         for _ in range(n):
193             image = tf.nn.conv2d(image, filters=kernel, strides=[1, 1, 1, \
194                               1], padding="SAME")
195             image = tf.squeeze(tf.squeeze(image, axis=0), axis=-1)
196         image = tf.clip_by_value(image, 0.0, 1.0)
197
198     return {
199         "image": image,
200         "segmentation_mask": example["segmentation_mask"]
201     }
202
203 def third_perturbation(example, a=1.0):
204     '''Contrast increase/decrease'''
205     image = example["image"]
206     image = tf.clip_by_value(image * a, 0.0, 1.0)
207
208     return {
209         "image": image,
210         "segmentation_mask": example["segmentation_mask"]
211     }
212
213 def fourth_perturbation(example, b=0.0):
214     '''Brightness increase/decrease'''
215     b = b / 255.0 # Normalize to [0, 1]
216     image = example["image"]
217     image = tf.clip_by_value(image + b, 0.0, 1.0)
218
219     return {
220         "image": image,
221         "segmentation_mask": example["segmentation_mask"]
222     }
223
224 def fifth_perturbation(example, p=0.0):
225     '''Occlusion of the Image Increase'''
226     image = example["image"]
227     shape = tf.shape(image)
228     height = shape[0]
229     width = shape[1]
230     if p == 0:
231         pass
232     else:
233         p = tf.cast(p, tf.int32)
234         y = tf.random.uniform(shape=[], minval=0, maxval=tf.maximum(1, \
235                               height - p), dtype=tf.int32)
236         x = tf.random.uniform(shape=[], minval=0, maxval=tf.maximum(1, \
237                               width - p), dtype=tf.int32)
238         p = tf.minimum(p, tf.minimum(height, width))
239         # Create meshgrid for the occlusion area
240         y_indices = tf.range(y, y + p)
241         x_indices = tf.range(x, x + p)
242         # Create grid coordinates for the occlusion
243         indices = tf.stack(tf.meshgrid(y_indices, x_indices, \
244                                       indexing='ij'), axis=-1)
245         indices = tf.reshape(indices, [-1, 2])

```

```

241     # Add channel dimension to indices
242     channel_indices = tf.tile(tf.expand_dims(indices, 1), [1, 3, 1])
243     channel_indices = tf.concat([
244         channel_indices[:, 0, :],
245         tf.expand_dims(tf.zeros_like(indices[:, 0]), 1)
246     ], axis=1)
247     channel_indices = tf.concat([
248         channel_indices[:, :2],
249         tf.expand_dims(tf.ones_like(indices[:, 0]), 1)
250     ], axis=1)
251     channel_indices = tf.concat([
252         channel_indices[:, :2],
253         tf.expand_dims(tf.ones_like(indices[:, 0]) + \
254             tf.ones_like(indices[:, 0]), 1)
255     ], axis=1)
256     mask = tf.ones_like(image)
257     zeros_patch = tf.zeros([p, p, 3])
258     indices = tf.stack([
259         tf.repeat(tf.range(y, y + p), p),
260         tf.tile(tf.range(x, x + p), [p])
261     ], axis=1)
262     zeros_flat = tf.zeros([p * p, 3])
263     mask = tf.tensor_scatter_nd_update(mask, indices, zeros_flat)
264     image = image * mask
265     return {
266         "image": image,
267         "segmentation_mask": example["segmentation_mask"]
268     }
269
270 def sixth_perturbation(example, d=0):
271     '''Salt and Pepper Noise'''
272     image = example["image"]
273     salt_mask = tf.random.uniform(tf.shape(image)) < (d / 2.0)
274     pepper_mask = tf.random.uniform(tf.shape(image)) < (d / 2.0)
275     noisy_image = tf.where(salt_mask, tf.ones_like(image), image)
276     noisy_image = tf.where(pepper_mask, tf.zeros_like(image), noisy_image)
277     return {
278         "image": noisy_image,
279         "segmentation_mask": example["segmentation_mask"]
280     }

```

D Augmentations Code

This section presents the TensorFlow-based augmentation functions used in this study, categorized into geometric transformations, colour modifications, Noise transformations, and their combinations. These augmentations enhance model robustness by increasing data variability during training.

D.1 Geometric Transformations

Geometric augmentations modify image orientation and positioning while preserving the overall content. These transformations help the model become invariant to spatial variations.

Listing 1: Geometric Augmentations

```

1  import tensorflow as tf
2  from tensorflow import keras
3  from tensorflow.keras import layers
4

```

```

5  # Define geometric augmentation layers for images
6  def augmentation_layers_geometric():
7      """Applies horizontal flips, translations, and rotations to images."""
8      return keras.Sequential([
9          layers.RandomFlip('horizontal'), # Flip image horizontally with a \
              50% probability
10         layers.RandomTranslation(height_factor=0.1, width_factor=0.1, \
              fill_mode='constant'), # Apply random shifts
11         layers.RandomRotation(factor=0.02, interpolation="nearest"), # \
              Small rotation to preserve structure
12     ])
13
14 # Define geometric augmentation layers for segmentation masks (no flipping)
15 def augmentation_layers_geometric_mask():
16     """Applies only translations and rotations to segmentation masks to \
        ensure label consistency."""
17     return keras.Sequential([
18         layers.RandomTranslation(height_factor=0.1, width_factor=0.1, \
              fill_mode='constant'),
19         layers.RandomRotation(factor=0.02, interpolation="nearest"),
20     ])

```

D.2 Colour Transformations

Colour-based augmentations alter brightness, contrast, saturation, and hue to simulate different lighting conditions and improve generalization.

Listing 2: Colour Augmentations

```

1  # Define colour augmentation layers
2  def augmentation_layers_colour():
3      """Applies random brightness, contrast, saturation, and hue shifts to \
        images."""
4      return keras.Sequential([
5          layers.RandomBrightness(factor=0.2, value_range=(0,1)), # Adjust \
              brightness randomly
6          layers.RandomContrast(factor=0.1), # Modify contrast levels
7          layers.Lambda(lambda x: random_saturation(x)), # Apply random \
              saturation changes
8          layers.Lambda(lambda x: random_hue(x)), # Shift hue values randomly
9      ])
10
11 # Apply random saturation adjustment
12 def random_saturation(image):
13     """Modifies the saturation of an image within a predefined range."""
14     return tf.image.random_saturation(image, lower=0.5, upper=1.5)
15
16 # Apply random hue shift
17 def random_hue(image):
18     """Shifts the hue of an image slightly to introduce colour \
        variations."""
19     return tf.image.random_hue(image, max_delta=0.05)

```

D.3 Noise and Filtering

To enhance robustness, noise perturbations and compression artifacts are applied. These augmentations help models become resilient to real-world applications.

Listing 3: Noise and Filtering

```

1 def augmentation_layers_noise_filter():
2     return keras.Sequential([
3         layers.GaussianNoise(0.1),
4         layers.Lambda(lambda x, training=None: random_blur(x)),
5         layers.Lambda(lambda x, training=None: random_jpeg_compression(x)),
6     ])
7
8 def random_blur(x):
9     """Randomly applies either a Gaussian blur or a motion blur."""
10    def apply_gaussian():
11        return gaussian_blur(x, kernel_size=3, sigma=1.0)
12    def apply_motion():
13        return motion_blur(x, kernel_size=3)
14    return tf.cond(tf.random.uniform([], 0, 1) < 0.5, apply_gaussian, \
15                  apply_motion)
16
17 def jpeg_compression_single_py(image, quality):
18     # Convert quality to a Python int.
19     quality = int(quality)
20     image_uint8 = tf.image.convert_image_dtype(image, tf.uint8)
21     jpeg = tf.io.encode_jpeg(image_uint8, quality=quality)
22     image_decoded = tf.io.decode_jpeg(jpeg, channels=3)
23     image_float = tf.image.convert_image_dtype(image_decoded, tf.float32)
24     return image_float

```

D.4 Applying Augmentations to Images and Masks

To maintain consistency in segmentation tasks, augmentations are dynamically applied to both images and masks. Geometric transformations modify both to ensure alignment, while Colour and Noise augmentations affect only the image. The updated implementation requires preserving the integrity of class labels in the segmentation mask. To avoid interpolation errors during transformations, the mask is processed using nearest-neighbor resizing, followed by a filtering step that restores only the original class values {0: 'background'}, {1: 'dog'}, {2: 'cat'}.

Listing 4: Applying Augmentations to Images and Masks

```

1 augmentation_colour = augmentation_layers_colour()
2 augmentation_geometric = augmentation_layers_geometric()
3 augmentation_noise_filter = augmentation_layers_noise_filter()
4
5 def augment_image_mask(image, mask, augment_colour=None, \
6   augment_geometric=None, augment_noise_filter=None):
7     """Applies selected augmentations to images and segmentation masks \
8       while preserving mask integrity."""
9     batched_image = tf.expand_dims(image, axis=0)
10    batched_mask = tf.expand_dims(mask, axis=0)
11
12    # Extract unique class values from the original mask
13    original_classes = tf.cast(tf.unique(tf.reshape(batched_mask, \
14      [-1]))[0], tf.uint8)
15
16    # Ensure mask remains categorical (discrete class values)
17    batched_mask = tf.cast(batched_mask, tf.uint8)
18
19    if augment_colour:
20        batched_image = augmentation_colour(batched_image, training=True)
21
22    if augment_geometric:
23        # Apply geometric transformations to both image and mask

```



```

21     concat_img_mask = tf.concat([batched_image, tf.cast(batched_mask, \
22         tf.float32)], axis=-1)
23     concat_img_mask = augmentation_layers_geometric()(concat_img_mask, \
24         training=True)
25
26     # Separate image and mask after transformations
27     batched_image, batched_mask = concat_img_mask[..., :-1], \
28         concat_img_mask[..., -1:]
29
30     # Use nearest-neighbor interpolation to preserve discrete class \
31     labels
32     batched_mask = tf.image.resize(
33         batched_mask, tf.shape(batched_mask)[1:3],
34         method=tf.image.ResizeMethod.NEAREST_NEIGHBOR
35     )
36
37     # Prevent class label corruption by rounding values
38     batched_mask = tf.round(batched_mask)
39     batched_mask = tf.cast(batched_mask, tf.uint8)
40
41     # Filter out any unintended values introduced by transformations
42     def filter_classes(mask, original_classes):
43         with tf.device('/CPU:0'): # Move operation to CPU for stability
44             mask_expanded = tf.expand_dims(mask, axis=-1) # Shape: \
45                 (H, W, 1)
46             original_classes_expanded = tf.reshape(original_classes, \
47                 [1, 1, -1]) # Shape: (1, 1, N_classes)
48
49             # Ensure dtype consistency
50             original_classes_expanded = \
51                 tf.cast(original_classes_expanded, tf.uint8)
52
53             # Create a boolean mask that checks against \
54             'original_classes'
55             is_valid = tf.reduce_any(tf.equal(mask_expanded, \
56                 original_classes_expanded), axis=-1)
57
58             # Convert boolean mask to uint8 for correct application
59             is_valid_uint8 = tf.cast(is_valid, tf.uint8)
60
61             # Apply filtering to retain only valid class values
62             valid_mask = tf.multiply(mask, is_valid_uint8)
63
64         return valid_mask
65
66     # Apply filtering to restore original class values
67     batched_mask = filter_classes(batched_mask, original_classes)
68
69     if augment_noise_filter:
70         batched_image = augmentation_layers_noise_filter()(batched_image, \
71             training=True)
72
73     return tf.squeeze(batched_image, axis=0), tf.squeeze(batched_mask, \
74         axis=0)
75
76 def augment_image_mask_2d(image, mask, augment_colour=None, \
77     augment_geometric=None, augment_noise_filter=None):
78     batched_image = tf.expand_dims(image, axis=0)
79     batched_mask = tf.expand_dims(mask, axis=0)
80     batched_mask = tf.cast(batched_mask, tf.float32)

```

```

69
70     if augment_colour:
71         batched_image = augmentation_colour(batched_image, training=True)
72
73     if augment_geometric:
74         combined = tf.concat([batched_image, batched_mask], axis=-1)
75         combined = augmentation_geometric(combined, training=True)
76         batched_image, batched_mask = tf.split(combined, [3, \
77             tf.shape(batched_mask)[-1]], axis=-1)
78
79     if augment_noise_filter:
80         batched_image = augmentation_noise_filter(batched_image, \
81             training=True)
82
83     image = tf.squeeze(batched_image, axis=0)
84     mask = tf.squeeze(batched_mask, axis=0)
85     mask = tf.cast(mask, tf.uint8)
86
87     # get heatmap for random points of animal
88     location = random_loc(mask_shape=mask.shape)
89     heatmap = create_gaussian_heatmap(mask_shape=mask.shape, loc=location, \
90         sigma=20)
91     mask = mask_heat_modification(mask, location)
92     image_heatmap = tf.concat([image, heatmap], axis=-1)
93
94     return image_heatmap, mask

```

D.5 Batch Processing with Augmentations

Augmentations are applied dynamically per batch during training, ensuring an efficient and diverse data feed. In the ‘batch’ function, augmentation can be specified as ‘none’, ‘colour’, ‘geometric’, ‘colour+geometric’, or ‘colour+geometric+noise_filter’

Listing 5: Batch Processing with Augmentations

```

1  def extract_image_mask(data):
2      """Extracts image and mask from dataset dictionary."""
3      return data['image'], data['segmentation_mask']
4
5  # Apply extraction to datasets
6  train = train.map(extract_image_mask)
7  val = val.map(extract_image_mask)
8  test = test.map(extract_image_mask)
9
10 BATCH_SIZE = 32
11
12 # Dynamically apply augmentation strategies during batching
13 train_ds = batch(train, augment='none', batch_size=BATCH_SIZE)
14 val_ds = batch(val, augment='none', batch_size=BATCH_SIZE)
15 test_ds = batch(test, augment='none', batch_size=BATCH_SIZE)

```

D.6 Batch function

The batching function efficiently processes datasets by dynamically applying augmentations within the tf.data pipeline. It utilizes caching for performance optimization, mapping for augmentation application, and prefetching to minimize data loading overhead. Augmentations are selectively applied: geometric transformations affect both images and masks, while Colour and Noise augmentations modify only images. Multi-threaded execution with tf.data.AUTOTUNE ensures efficient parallel processing, optimizing computational performance during training.

Listing 6: Efficient Batching with Augmentation selection

```

1 def batch(data, augment='none', batch_size=32):
2     if augment == 'none':
3         # Return dataset without augmentation
4         return data.batch(batch_size=batch_size).prefetch(tf.data.AUTOTUNE)
5
6     # Cache dataset for efficiency
7     cached_train = data.cache()
8
9     if augment == 'colour':
10        return cached_train.map(
11            lambda image, mask: augment_image_mask(
12                image, mask, augment_colour=True,
13                augment_geometric=False, augment_noise_filter=False
14            ),
15            num_parallel_calls=tf.data.AUTOTUNE
16        ).batch(batch_size).prefetch(tf.data.AUTOTUNE)
17
18    if augment == 'geometric':
19        return cached_train.map(
20            lambda image, mask: augment_image_mask(
21                image, mask, augment_colour=False,
22                augment_geometric=True, augment_noise_filter=False
23            ),
24            num_parallel_calls=tf.data.AUTOTUNE
25        ).batch(batch_size).prefetch(tf.data.AUTOTUNE)
26
27    if augment == 'colour+geometric':
28        return cached_train.map(
29            lambda image, mask: augment_image_mask(
30                image, mask, augment_colour=True,
31                augment_geometric=True, augment_noise_filter=False
32            ),
33            num_parallel_calls=tf.data.AUTOTUNE
34        ).batch(batch_size).prefetch(tf.data.AUTOTUNE)
35
36    if augment == 'colour+geometric+noise_filter':
37        return cached_train.map(
38            lambda image, mask: augment_image_mask(
39                image, mask, augment_colour=True,
40                augment_geometric=True, augment_noise_filter=True
41            ),
42            num_parallel_calls=tf.data.AUTOTUNE
43        ).batch(batch_size).prefetch(tf.data.AUTOTUNE)
44
45
46    if augment == 'training_2d':
47        cached_train = data.cache()
48        return cached_train.map(
49            lambda image, mask: augment_image_mask_2d(image, mask, \
50                augment_colour=True, augment_geometric=True, \
51                augment_noise_filter=False),
52            num_parallel_calls=tf.data.AUTOTUNE
53        ).batch(batch_size).prefetch(tf.data.AUTOTUNE)

```

E Networks design

The code below, describes the architecture, implementation details of each of the segmentation models, U-Net, Autoencoder pre-training for segmentation, CLIP Segmentation Model

E.1 U-Net Model

The U-Net model is a convolutional neural network for image segmentation, featuring an encoder that compresses input through convolutions and pooling, and a decoder that reconstructs it using transposed convolutions and skip connections, outputting a segmented image.

```
1 def UNet_model(input_shape=(128, 128, 3)):
2     inputs = tf.keras.Input(shape=input_shape)
3
4     # Encoder
5     conv1 = layers.Conv2D(64, 3, activation='relu', padding='same')(inputs)
6     conv1 = layers.Conv2D(64, 3, activation='relu', padding='same')(conv1)
7     pool1 = layers.MaxPooling2D(pool_size=(2, 2))(conv1)
8
9     conv2 = layers.Conv2D(128, 3, activation='relu', padding='same')(pool1)
10    conv2 = layers.Conv2D(128, 3, activation='relu', padding='same')(conv2)
11    pool2 = layers.MaxPooling2D(pool_size=(2, 2))(conv2)
12
13    conv3 = layers.Conv2D(256, 3, activation='relu', padding='same')(pool2)
14    conv3 = layers.Conv2D(256, 3, activation='relu', padding='same')(conv3)
15    pool3 = layers.MaxPooling2D(pool_size=(2, 2))(conv3)
16
17    conv4 = layers.Conv2D(512, 3, activation='relu', padding='same')(pool3)
18    conv4 = layers.Conv2D(512, 3, activation='relu', padding='same')(conv4)
19    pool4 = layers.MaxPooling2D(pool_size=(2, 2))(conv4)
20
21    # Bottleneck
22    conv5 = layers.Conv2D(1024, 3, activation='relu', padding='same')(pool4)
23    conv5 = layers.Conv2D(1024, 3, activation='relu', padding='same')(conv5)
24
25    # Decoder
26    up6 = layers.Conv2DTranspose(512, 2, strides=(2, 2), \
27        padding='same')(conv5)
28    up6 = layers.concatenate([up6, conv4])
29    conv6 = layers.Conv2D(512, 3, activation='relu', padding='same')(up6)
30    conv6 = layers.Conv2D(512, 3, activation='relu', padding='same')(conv6)
31
32    up7 = layers.Conv2DTranspose(256, 2, strides=(2, 2), \
33        padding='same')(conv6)
34    up7 = layers.concatenate([up7, conv3])
35    conv7 = layers.Conv2D(256, 3, activation='relu', padding='same')(up7)
36    conv7 = layers.Conv2D(256, 3, activation='relu', padding='same')(conv7)
37
38    up8 = layers.Conv2DTranspose(128, 2, strides=(2, 2), \
39        padding='same')(conv7)
40    up8 = layers.concatenate([up8, conv2])
41    conv8 = layers.Conv2D(128, 3, activation='relu', padding='same')(up8)
42    conv8 = layers.Conv2D(128, 3, activation='relu', padding='same')(conv8)
43
44    up9 = layers.Conv2DTranspose(64, 2, strides=(2, 2), \
45        padding='same')(conv8)
46    up9 = layers.concatenate([up9, conv1])
47    conv9 = layers.Conv2D(64, 3, activation='relu', padding='same')(up9)
48    conv9 = layers.Conv2D(64, 3, activation='relu', padding='same')(conv9)
49
50    #Output
51    outputs = layers.Conv2D(3, 1, activation='softmax')(conv9)
52    model = models.Model(inputs, outputs)
53    return model
```

E.2 Autoencoder Model

The Autoencoder consists of two main components: an encoder that compresses the input images into a feature-rich representation and a decoder that reconstructs the original images from this representation.

Listing 7: Autoencoder Model

```
1 import tensorflow as tf
2 from tensorflow.keras import layers, models
3
4 class Autoencoder(tf.keras.Model):
5     def __init__(self, input_shape=(256, 256, 3)):
6         super(Autoencoder, self).__init__()
7         self.encoder = models.Sequential([
8             layers.Conv2D(64, (3, 3), activation='relu', padding='same', \
9                 input_shape=input_shape),
10            layers.MaxPooling2D((2, 2), padding='same'),
11            layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
12            layers.MaxPooling2D((2, 2), padding='same'),
13            layers.Conv2D(256, (3, 3), activation='relu', padding='same'),
14            layers.MaxPooling2D((2, 2), padding='same'),
15            layers.Conv2D(512, (3, 3), activation='relu', padding='same'),
16            layers.MaxPooling2D((2, 2), padding='same'),
17            layers.Conv2D(1024, (3, 3), activation='relu', padding='same')
18        ])
19
20        self.decoder = models.Sequential([
21            layers.Conv2DTranspose(512, (3, 3), activation='relu', \
22                padding='same'),
23            layers.UpSampling2D((2, 2)),
24            layers.Conv2DTranspose(256, (3, 3), activation='relu', \
25                padding='same'),
26            layers.UpSampling2D((2, 2)),
27            layers.Conv2DTranspose(128, (3, 3), activation='relu', \
28                padding='same'),
29            layers.UpSampling2D((2, 2)),
30            layers.Conv2DTranspose(64, (3, 3), activation='relu', \
31                padding='same'),
32            layers.UpSampling2D((2, 2)),
33            layers.Conv2D(3, (3, 3), activation='sigmoid', padding='same')
34        ])
35
36    def call(self, x):
37        encoded = self.encoder(x)
38        decoded = self.decoder(encoded)
39        return decoded
```

E.2.1 Autoencoder pre-training for segmentation

The segmentation model enables the pre-trained encoder from the Autoencoder to enhance feature extraction. It uses transposed convolutional layers to progressively upscale the features back to the original resolution, concluding with a softmax layer for pixel-wise segmentation.

Listing 8: Segmentation Model using Pre-trained Encoder

```
1 def build_segmentation_decoder(encoder, num_classes=3):
2     inputs = layers.Input(shape=(256, 256, 3))
3     x = encoder(inputs, training=False)
```

```

4     x = layers.Conv2DTranspose(512, (3, 3), activation='relu', \
        padding='same')(x)
5     x = layers.UpSampling2D((2, 2))(x)
6     x = layers.Conv2DTranspose(256, (3, 3), activation='relu', \
        padding='same')(x)
7     x = layers.UpSampling2D((2, 2))(x)
8     x = layers.Conv2DTranspose(128, (3, 3), activation='relu', \
        padding='same')(x)
9     x = layers.UpSampling2D((2, 2))(x)
10    x = layers.Conv2DTranspose(64, (3, 3), activation='relu', \
        padding='same')(x)
11    x = layers.UpSampling2D((2, 2))(x)
12    outputs = layers.Conv2D(num_classes, 1, activation='softmax', \
        padding='same')(x)
13    return models.Model(inputs, outputs)

```

E.3 CLIP Segmentation Model

This model utilizes a CLIP Vision Transformer as the encoder and a convolutional neural network as the decoder.

E.3.1 CLIP Encoder Layer

The encoder layer uses the pre-trained CLIP Vision Transformer model to generate feature representations.

Listing 9: Implementation of CLIP Encoder Layer

```

1  import tensorflow as tf
2  from transformers import TFCLIPVisionModel
3
4  class CLIPEncoderLayer(tf.keras.layers.Layer):
5      def __init__(self, **kwargs):
6          super().__init__(**kwargs)
7          self.clip_encoder = \
8              TFCLIPVisionModel.from_pretrained("openai/clip-vit-base-patch32")
9          self.clip_encoder.trainable = False
10
11     def call(self, inputs):
12         pixel_values = tf.transpose(inputs, [0, 3, 1, 2]) # NHWC to NCHW
13         outputs = self.clip_encoder(pixel_values=pixel_values)
14         return outputs.last_hidden_state
15
16     def compute_output_shape(self, input_shape):
17         batch_size = input_shape[0]
18         return batch_size, 50, 768

```

E.3.2 Full Segmentation Model

This section integrates the encoder layer into a complete segmentation model.

Listing 10: Full segmentation model using CLIP

```

1  from tensorflow.keras.layers import Input, Resizing, Lambda, Conv2D, \
    UpSampling2D, Reshape
2  from tensorflow.keras.models import Model
3

```

```

4 def clip_segmentation_model(input_shape=(128, 128, 3)):
5     input_img = Input(shape=input_shape, name="input_image")
6
7     x = Resizing(224, 224, name="resize_input")(input_img)
8
9     def clip_normalize(x):
10         mean = [0.48145466, 0.4578275, 0.40821073]
11         std = [0.26862954, 0.26130258, 0.27577711]
12         return tf.stack([(x[..., i] - mean[i]) / std[i] for i in \
13             range(3)], axis=-1)
14
15     x = Lambda(clip_normalize, name="clip_normalization")(x)
16
17     clip_features = CLIPEncoderLayer()(x)
18
19     patch_features = Lambda(lambda x: x[:, 1:, :], \
20         name="remove_cls_token")(clip_features)
21     features = Reshape((7, 7, 768), name="reshape_features")(patch_features)
22
23     x = Conv2D(256, 3, padding='same', activation='relu')(features)
24     x = UpSampling2D((2, 2))(x)
25     x = Conv2D(128, 3, padding='same', activation='relu')(x)
26     x = UpSampling2D((2, 2))(x)
27     x = Conv2D(64, 3, padding='same', activation='relu')(x)
28     x = UpSampling2D((2, 2))(x)
29     x = Conv2D(32, 3, padding='same', activation='relu')(x)
30     x = UpSampling2D((2, 2))(x)
31
32     outputs = Conv2D(3, 1, activation='softmax')(x)
33
34     if input_shape[:2] != (224, 224):
35         outputs = Resizing(input_shape[0], input_shape[1])(outputs)
36
37     return Model(inputs=input_img, outputs=outputs, \
38         name="CLIP_Segmentation")

```

F Training Pipeline and Model Calls

F.1 Directory Structure and Experiment Runs

Each model is assigned a unique top-level directory reflecting its experiment run:

- **task_2a/baseline:** U-Net, the baseline model, trained without augmentation to establish a reference performance.
- **task_2a/augmented:** U-Net trained with different augmentation strategies (colour, geometric, colour+geometric, colour+geometric+noise).
- **task_2b/baseline:** Autoencoder-pretrained segmentation model, using an encoder pre-trained with MSE, trained with augmented data.
- **task_2b/augmented:** Autoencoder-pretrained model trained under different augmentation strategies (colour, geometric, colour+geometric, colour+geometric+noise).
- **task_2c/baseline:** CLIP-based segmentation model, leveraging CLIP features, trained with augmented data.
- **task_2c/augmented:** CLIP-based segmentation model trained with different augmentation strategies (colour, geometric, colour+geometric, colour+geometric+noise).

- **task_2d/colour+geometric**: Prompt-based CLIP segmentation model, incorporating user clicks for binary segmentation, trained using the best augmentation technique identified from CLIP experiments (colour+geometric).

The **baseline** suffix indicates the initial experiment setup, while **augmented** directories store models trained with various augmentation strategies. The Prompt-based CLIP segmentation model is trained using only the best-performing augmentation strategy (colour+geometric) as identified from CLIP model experiments. Augmentation techniques are applied to training sets, as detailed in Section Data Augmentations.

F.2 Training Pipeline

The following code provides a common training pipeline for initializing, compiling, and training segmentation models.

Listing 11: Common Training Pipeline Functions

```

1  # pipeline.py: Common Training Pipeline Functions
2
3  import tensorflow as tf
4  from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping
5  from metrics import MeanIoUWrapper, dice_coefficient # Custom metrics
6  import os
7
8  def initialize_and_compile_model(
9      model_instance,          # Pre-constructed model instance
10     top_dir,                  # Base directory
11     model_name,               # Model folder name
12     num_classes=3,           # Number of classes (None for \
                               binary/non-segmentation)
13     loss='sparse_categorical_crossentropy', # Loss function
14     use_segmentation_metrics=True        # Flag for segmentation metrics
15 ):
16     """Compile a pre-constructed model with specified loss and metrics."""
17     best_model_folder = os.path.join(top_dir, model_name)
18     metrics = ['accuracy']
19     if use_segmentation_metrics and num_classes:
20         miou_metric = MeanIoUWrapper(num_classes=num_classes)
21         metrics.extend([miou_metric, dice_coefficient])
22     model_instance.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4, \
        beta_1=0.9, beta_2=0.999),
23                           loss=loss, metrics=metrics)
24     return model_instance, best_model_folder
25
26 def train_model(
27     model,                    # Compiled model
28     best_model_folder,        # Save path
29     train_ds,                 # Training dataset
30     val_ds,                   # Validation dataset
31     epochs=100,               # Max epochs
32     patience=20               # Early stopping patience
33 ):
34     """Train the model with early stopping and checkpointing."""
35     checkpoint = ModelCheckpoint(
36         best_model_folder + '.keras', monitor='val_loss', \
37         save_best_only=True,
38         mode='min', verbose=1
39     )
40     early_stopping = EarlyStopping(monitor='val_loss', patience=patience,
        restore_best_weights=True)

```

```

41     history = model.fit(train_ds, epochs=epochs, validation_data=val_ds,
42                         callbacks=[early_stopping, checkpoint])
43     return history

```

F.3 Model Construction and Training Calls

The following code defines model construction and training procedures for each segmentation model.

Listing 12: Model Construction and Training Calls

```

1  # model_calls.py: Model Construction and Training Calls
2
3  import tensorflow as tf
4  from pipeline import initialize_and_compile_model, train_model
5  from models import UNet_model, CLIP_model, Prompt_CLIP_model, \
6      build_segmentation_decoder
7  import os
8
9  # 1. UNet Construction and Training
10 unet_top_dir = "task_2a/baseline" # Baseline model, no augmentation
11 unet_model = UNet_model(input_shape=reshape_size + (3,))
12 unet_model, unet_folder = initialize_and_compile_model(
13     model_instance=unet_model, top_dir=unet_top_dir, model_name="Unet_model"
14 )
15 history_unet = train_model(unet_model, unet_folder, train_ds, val_ds)
16
17 # 2. Autoencoder-Pretrained Segmentation Model Construction and Training
18 pretrained_top_dir = "task_2b/baseline" # Second experiment run
19 encoder = tf.keras.models.load_model(os.path.join(pretrained_top_dir, \
20     "autoencoder_models/encoder_training_tf"))
21 encoder.load_weights(os.path.join(pretrained_top_dir, \
22     "autoencoder_weights/encoder_weights.h5"))
23 pretrained_seg_model = build_segmentation_decoder(encoder, num_classes=3)
24 pretrained_model, pretrained_folder = initialize_and_compile_model(
25     model_instance=pretrained_seg_model, top_dir=pretrained_top_dir,
26     model_name="autoencoder_pretrained_model"
27 )
28 history_pretrained = train_model(pretrained_model, pretrained_folder, \
29     train_ds, val_ds)
30
31 # 3. CLIP-based Segmentation Model Construction and Training
32 clip_top_dir = "task_2c/baseline" # Third experiment run
33 clip_model = CLIP_model(input_shape=reshape_size + (3,))
34 clip_model, clip_folder = initialize_and_compile_model(
35     model_instance=clip_model, top_dir=clip_top_dir, \
36     model_name="clip_segmentation_model"
37 )
38 history_clip = train_model(clip_model, clip_folder, train_ds, val_ds)
39
40 # 4. Prompt-based CLIP Segmentation Model Construction and Training
41 prompt_top_dir = "task_2d/colour+geometric" # Single experiment \
42     configuration
43 prompt_model = Prompt_CLIP_model(input_shape=reshape_size + (3,))
44 prompt_model, prompt_folder = initialize_and_compile_model(
45     model_instance=prompt_model, top_dir=prompt_top_dir, \
46     model_name="Prompt_CLIP_model",
47     num_classes=2, loss='binary_crossentropy', \
48     use_segmentation_metrics=False
49 )

```

```
42 history_prompt = train_model(prompt_model, prompt_folder, train_ds, val_ds)
```

F.4 Metrics Implementations

To evaluate segmentation performance, we use two key metrics: Mean Intersection over Union (Mean IoU) and Dice Coefficient. These metrics assess how well the predicted segmentation masks align with the ground truth labels.

Mean IoU measures the average intersection over union across all classes, computed as:

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|}, \quad (1)$$

where A is the predicted mask and B is the ground truth mask. This metric provides an overall assessment of segmentation accuracy per class.

Dice Coefficient quantifies the overlap between predicted and ground truth masks, emphasizing both precision and recall:

$$\text{Dice} = \frac{2|A \cap B|}{|A| + |B|}. \quad (2)$$

The following code defines the implementations of these metrics:

Listing 13: Metrics Implementations

```
1  # Metrics Implementations
2
3  import tensorflow as tf
4  from tensorflow.keras import backend as K
5
6  class MeanIoUWrapper(tf.keras.metrics.MeanIoU):
7      """Custom wrapper for MeanIoU metric to handle multi-class \
      segmentation."""
8      def __init__(self, num_classes, name='mean_iou', **kwargs):
9          super().__init__(num_classes=num_classes, name=name, **kwargs)
10         self.num_classes = num_classes
11
12     def update_state(self, y_true, y_pred, sample_weight=None):
13         if y_pred.shape[-1] == 1:
14             y_pred = tf.cast(y_pred > 0.5, tf.int32)
15         else:
16             y_pred = tf.argmax(y_pred, axis=-1)
17             if tf.shape(y_true).shape.rank > tf.shape(y_pred).shape.rank:
18                 y_true = tf.squeeze(y_true, axis=-1)
19             y_true = tf.cast(y_true, tf.int32)
20             return super().update_state(y_true, y_pred, sample_weight)
21
22     def get_config(self):
23         config = super().get_config()
24         config.update({'num_classes': self.num_classes})
25         return config
26
27 def dice_coefficient(y_true, y_pred, num_classes=None, threshold=0.5):
28     """Compute Dice Coefficient for binary or multi-class segmentation."""
29     y_true = tf.cast(y_true, tf.float32)
30     if num_classes is None or num_classes == 1:
31         y_pred_bin = tf.cast(y_pred > threshold, tf.float32)
32         y_true_bin = tf.squeeze(y_true, axis=-1) if y_true.shape[-1] == 1 \
            else y_true
33         intersection = tf.reduce_sum(y_true_bin * y_pred_bin)
```

```

34         union = tf.reduce_sum(y_true_bin) + tf.reduce_sum(y_pred_bin)
35         dice = (2.0 * intersection + K.epsilon()) / (union + K.epsilon())
36         return dice
37     else:
38         y_pred = tf.argmax(y_pred, axis=-1)
39         y_true = tf.cast(tf.squeeze(y_true, axis=-1), tf.int32) if \
40             y_true.shape[-1] == 1 else tf.cast(y_true, tf.int32)
41         y_true_one_hot = tf.one_hot(y_true, depth=num_classes, \
42             dtype=tf.float32)
43         y_pred_one_hot = tf.one_hot(y_pred, depth=num_classes, \
44             dtype=tf.float32)
45         intersection = K.sum(y_true_one_hot * y_pred_one_hot, axis=[0, 1, \
46             2])
47         union = K.sum(y_true_one_hot, axis=[0, 1, 2]) + \
48             K.sum(y_pred_one_hot, axis=[0, 1, 2])
49         dice_per_class = (2.0 * intersection) / (union + K.epsilon())
50         return K.mean(dice_per_class)

```

G Evaluation Code

```

1 def evaluate_model(model, test_data, test_labels):
2     loss, accuracy = model.evaluate(test_data, test_labels)
3     print(f"Test Accuracy: {accuracy:.2f}")

```

H Prompt-based segmentation

H.1 Prompt-based segmentation utility functions Code

These utility functions support prompt-based segmentation by generating random point locations within a mask and creating Gaussian heatmaps centered at specified coordinates, alongside modifying masks based on regional labels at those points.

```

1 def random_loc(mask_shape):
2     height, width, _ = mask_shape
3     y = tf.random.uniform((), 0, height, dtype=tf.int32)
4     x = tf.random.uniform((), 0, width, dtype=tf.int32)
5     return tf.stack([y, x])
6
7 def create_gaussian_heatmap(mask_shape, loc, sigma=10.0):
8     height, width, channels = mask_shape
9
10    y_range = tf.range(height, dtype=tf.float32)
11    x_range = tf.range(width, dtype=tf.float32)
12    y_grid, x_grid = tf.meshgrid(y_range, x_range, indexing='ij')
13
14    # Swap the order: point[0] is y, point[1] is x.
15    y_point = tf.cast(loc[0], tf.float32)
16    x_point = tf.cast(loc[1], tf.float32)
17
18    # Calculate gaussian heatmap
19    squared_dist = tf.square(x_grid - x_point) + tf.square(y_grid - y_point)
20    heatmap = tf.exp(-squared_dist / (2.0 * sigma * sigma))
21
22    heatmap = tf.reshape(heatmap, (height, width, channels))
23    return heatmap
24
25 def mask_heat_modification(mask, loc):
26     loc_label = mask[loc[0], loc[1], 0]

```

```

27     region_interest_mask = tf.equal(mask, loc_label)
28     return tf.cast(region_interest_mask, tf.int32)

```

H.2 Prompt-based segmentation augmentation functions Code

This augmentation function enhances 2D image-mask pairs with optional colour, geometric, and noise transformations, then appends a Gaussian heatmap based on a random mask location to the image, returning the modified image-heatmap and mask.

```

1  def augment_image_mask_2d(image, mask, augment_colour=None, \
    augment_geometric=None, augment_noise_filter=None):
2      batched_image = tf.expand_dims(image, axis=0)
3      batched_mask = tf.expand_dims(mask, axis=0)
4      batched_mask = tf.cast(batched_mask, tf.float32)
5
6      if augment_colour:
7          batched_image = augmentation_colour(batched_image, training=True)
8
9      if augment_geometric:
10         combined = tf.concat([batched_image, batched_mask], axis=-1)
11         combined = augmentation_geometric(combined, training=True)
12         batched_image, batched_mask = tf.split(combined, [3, \
            tf.shape(batched_mask)[-1]], axis=-1)
13
14     if augment_noise_filter:
15         batched_image = augmentation_noise_filter(batched_image, \
            training=True)
16
17     image = tf.squeeze(batched_image, axis=0)
18     mask = tf.squeeze(batched_mask, axis=0)
19     mask = tf.cast(mask, tf.uint8)
20
21     # get heatmap for random points of animal
22     location = random_loc(mask_shape=mask.shape)
23     heatmap = create_gaussian_heatmap(mask_shape=mask.shape, loc=location, \
        sigma=20)
24     mask = mask_heat_modification(mask, location)
25     image_heatmap = tf.concat([image, heatmap], axis=-1)
26
27     return image_heatmap, mask

```

H.3 CLIP Modified Model for prompt-based segmentation

The CLIP-modified model integrates a pre-trained CLIP encoder with a segmentation pipeline, processing input images and heatmaps through resizing, normalization, and convolutional layers, merging features for upsampled segmentation output.

```

1  import tensorflow as tf
2  from tensorflow.keras.layers import Input, Lambda, Resizing, Conv2D, \
    MaxPooling2D, UpSampling2D, Concatenate, Reshape
3  from tensorflow.keras.models import Model
4  from models import CLIPEncoderLayer
5
6  def clip_segmentation_model_2d(input_shape=(128, 128, 4)):
7      input_total = Input(shape=input_shape, name="input_total")
8      input_img = Lambda(lambda x: x[..., :3], \
        name="split_image")(input_total)
9      input_heatmap = Lambda(lambda x: x[..., 3:], \
        name="split_heatmap")(input_total)

```

```

10     x = Resizing(224, 224, name="resize_image")(input_img)
11     def clip_normalize(x):
12         mean = [0.48145466, 0.4578275, 0.40821073]
13         std = [0.26862954, 0.26130258, 0.27577711]
14         return tf.stack([(x[..., i] - mean[i]) / std[i] for i in \
15                             range(3)], axis=-1)
16     x = Lambda(clip_normalize, name="clip_normalization")(x)
17     clip_features = CLIPEncoderLayer()(x)
18     patch_features = Lambda(lambda x: x[:, 1:, :], \
19                             name="remove_cls_token")(clip_features)
20     features = Reshape((7, 7, 768), name="reshape_features")(patch_features)
21     heatmap_encoded = Resizing(28, 28, \
22                             name="heatmap_resize_initial")(input_heatmap)
23     heatmap_encoded = Conv2D(32, (3, 3), padding='same', \
24                             activation='relu', name="heatmap_conv1")(heatmap_encoded)
25     heatmap_encoded = MaxPooling2D(pool_size=(2, 2), \
26                                     name="heatmap_pool1")(heatmap_encoded)
27     heatmap_encoded = Conv2D(64, (3, 3), padding='same', \
28                             activation='relu', name="heatmap_conv2")(heatmap_encoded)
29     heatmap_encoded = MaxPooling2D(pool_size=(2, 2), \
30                                     name="heatmap_pool2")(heatmap_encoded)
31     heatmap_encoded = Conv2D(768, (1, 1), activation='relu', \
32                             name="heatmap_projection")(heatmap_encoded)
33     heatmap_encoded = Resizing(7, 7, \
34                             name="heatmap_resize_final")(heatmap_encoded)
35     features_combined = Concatenate(axis=-1, \
36                                     name="features_heatmap_merge")([features, heatmap_encoded])
37     x = Conv2D(256, 3, padding='same', activation='relu')(features_combined)
38     x = UpSampling2D((2, 2))(x)
39     x = Conv2D(128, 3, padding='same', activation='relu')(x)
40     x = UpSampling2D((2, 2))(x)
41     x = Conv2D(64, 3, padding='same', activation='relu')(x)
42     x = UpSampling2D((2, 2))(x)
43     x = Conv2D(32, 3, padding='same', activation='relu')(x)
44     x = UpSampling2D((2, 2))(x)
45     outputs = Conv2D(1, 1, activation='sigmoid')(x)
46     if input_shape[:2] != (224, 224):
47         outputs = Resizing(input_shape[0], input_shape[1])(outputs)
48     return Model(inputs=input_total, outputs=outputs, \
49                 name="CLIP_Segmentation_with_Heatmap")

```

H.4 Prompt-based segmentation UI Code

This UI code implements an interactive segmentation tool using OpenCV, loading a CLIP-based model to predict masks from user mouse clicks, overlaying results on the input image with a heatmap-guided visualization.

```

1  import cv2
2  import numpy as np
3  from point_segmentation import create_gaussian_heatmap
4  import tensorflow as tf
5  import os
6  import tensorflow as tf
7  from models import (
8      split_image,
9      split_heatmap,
10     clip_normalize,
11     remove_cls_token,
12     CLIPEncoderLayer
13 )

```

```

14
15 model_path = "Clip_model.keras"
16
17 custom_objects = {
18     "split_image": split_image,
19     "split_heatmap": split_heatmap,
20     "clip_normalize": clip_normalize,
21     "remove_cls_token": remove_cls_token,
22     "CLIPEncoderLayer": CLIPEncoderLayer
23 }
24
25 try:
26     model = tf.keras.models.load_model(
27         model_path,
28         custom_objects=custom_objects,
29         compile=False,
30         safe_mode=False
31     )
32     print("Model loaded successfully!")
33
34 except Exception as e:
35     print(f"Error loading model: {e}")
36
37 image_path = \
38     "/home/noxiusk/Desktop/data_science/cv/CV_Assignment/src/output_image8.png"
39 image = cv2.imread(image_path)
40 if image is None:
41     raise FileNotFoundError(f"Image not found at {image_path}")
42 image = image / 255.0
43 pred_mask_global = None
44
45 def mouse_callback(event, x, y, flags, param):
46     global pred_mask_global
47     if event == cv2.EVENT_LBUTTONDOWN:
48         print(f"Clicked at ({x}, {y})")
49         # Generate the gaussian heatmap
50         heatmap = create_gaussian_heatmap(mask_shape=(image.shape[0], \
51             image.shape[1], 1),
52             loc=(y, x), sigma=10)
53         input_data = np.concatenate([image, heatmap], axis=-1)
54         input_data = np.expand_dims(input_data, axis=0)
55
56         # Predict the mask
57         pred_mask = model.predict(input_data, verbose=0)
58         pred_mask = np.squeeze(pred_mask) # shape: (H, W)
59         pred_mask = (pred_mask > 0.5).astype(np.uint8) * 255
60         pred_mask_global = pred_mask
61
62 # Create a single window with no toolbar and bind the mouse callback
63 cv2.namedWindow("Image", cv2.WINDOW_AUTOSIZE | cv2.WINDOW_GUI_NORMAL)
64 cv2.setMouseCallback("Image", mouse_callback)
65 print("Click on the image window to generate a mask. Press 'ESC' or close \
66     the window to exit.")
67
68 while True:
69     display_img = (image * 255).astype(np.uint8).copy()
70
71     # If a mask has been generated, overlay it with a colourmap
72     if pred_mask_global is not None:
73         coloured_mask = cv2.applyColourMap(pred_mask_global, \

```

```
        cv2.COLORMAP_JET)
71     display_img = cv2.addWeighted(display_img, 0.7, coloured_mask, \
        0.3, 0)
72
73     cv2.imshow("Image", display_img)
74     key = cv2.waitKey(1)
75     if key == 27 or cv2.getWindowProperty("Image", cv2.WND_PROP_VISIBLE) \
        <= 0:
76         break
77
78 cv2.destroyAllWindows()
```
